0. State-space search: Water jug

```
import java.util.*;
public class Assignment0 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter jug A capacity, jug B capacity, target: ");
        int a = sc.nextInt(), b = sc.nextInt(), target = sc.nextInt();
        Queue<int[]> q = new LinkedList<>();
        Set<String> seen = new HashSet<>();
        q.add(new int[]{0, 0});
        while (!q.isEmpty()) {
            int[] s = q.remove();
            String key = s[0] + "," + s[1];
            if (!seen.add(key)) continue;
            System.out.println("(" + s[0] + ", " + s[1] + ")");
            if (s[0] == target || s[1] == target) {
                System.out.println("Target reached.");
                break;
            }
            int pour = Math.min(s[0], b - s[1]);
            q.add(new int[]{a, s[1]});                 // fill A
            q.add(new int[]{s[0], b});                 // fill B
            q.add(new int[]{0, s[1]});                 // empty A
            q.add(new int[]{s[0], 0});                 // empty B
            q.add(new int[]{s[0] - pour, s[1] + pour}); // pour A to B
        }
    }
}
```

1. A* algorithm: 8-puzzle

```
import java.util.*;
public class Assignment1 {
    static String goal = "123456780";
    static int h(String s) {
        int sum = 0;
        for (int i = 0; i < 9; i++) {
            int n = s.charAt(i) - '0';
            if (n != 0) {
                int g = n - 1;
                sum += Math.abs(i / 3 - g / 3) + Math.abs(i % 3 - g % 3);
            }
        }
        return sum;
    }
    static class Node {
        String state;
        int moves;
        Node(String s, int m) { state = s; moves = m; }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter 9 numbers (0 is blank):");
        String start = "";
        for (int i = 0; i < 9; i++) start += sc.next();
        PriorityQueue<Node> pq = new PriorityQueue<>(
            Comparator.comparingInt(n -> n.moves + h(n.state)));
        Map<String, Integer> best = new HashMap<>();
        Map<String, String> parent = new HashMap<>();
        pq.add(new Node(start, 0));
```

```
best.put(start, 0);
while (!pq.isEmpty()) {
    Node cur = pq.remove();
    if (cur.state.equals(goal)) break;
    int z = cur.state.indexOf('0');
    for (int m : new int[]{-3, 3, -1, 1}) {
        int k = z + m;
        if (k < 0 || k > 8 || (m == -1 && z % 3 == 0)
                           || (m == 1 && z % 3 == 2)) continue;
        char[] c = cur.state.toCharArray();
        c[z] = c[k]; c[k] = '0';
        String next = new String(c);
        if (cur.moves + 1 < best.getOrDefault(next, 9999)) {
            best.put(next, cur.moves + 1);
            parent.put(next, cur.state);
            pq.add(new Node(next, cur.moves + 1));
        }
    }
}
if (!best.containsKey(goal)) {
    System.out.println("No solution found.");
    return;
}
List<String> path = new ArrayList<>();
for (String s = goal; s != null; s = parent.get(s)) path.add(s);
Collections.reverse(path);
System.out.println("Solution in " + (path.size() - 1) + " moves:");
for (String s : path)
    System.out.println(s.substring(0, 3) + "\n" +
```

```
                                s.substring(3, 6) + "\n" + s.substring(6) + "\n");
    }
}
```

2. Hill climbing: Travelling Salesman

```
import java.util.*;
public class Assignment2 {
    static int n;
    static int[][] d;
    static int distance(int[] route) {
        int total = 0;
        for (int i = 0; i < n; i++)
            total += d[route[i]][route[(i + 1) % n]];
        return total;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of cities: ");
        n = sc.nextInt();
        d = new int[n][n];
        System.out.println("Enter distance matrix:");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) d[i][j] = sc.nextInt();
        int[] route = new int[n];
        for (int i = 0; i < n; i++) route[i] = i;
        boolean changed = true;
        while (changed) {
            changed = false;
            int oldDistance = distance(route);
            for (int i = 1; i < n; i++) {
                for (int j = i + 1; j < n; j++) {
                    int temp = route[i]; route[i] = route[j]; route[j] = temp;
                    if (distance(route) < oldDistance) {
                        oldDistance = distance(route);
```

```
                        changed = true;
                    } else {
                        temp = route[i]; route[i] = route[j]; route[j] = temp;
                    }
                }
            }
        }
        System.out.print("Route: ");
        for (int city : route) System.out.print(city + " ");
        System.out.println(route[0]);
        System.out.println("Total distance: " + distance(route));
    }
}
```

3. Constraint satisfaction: SEND + MORE = MONEY

```
public class Assignment3 {
    static int[] d = new int[8];
    static boolean[] used = new boolean[10];
    static boolean solve(int i) {
        if (i == 8)
            return d[0] != 0 && d[4] != 0 &&
                (1000*d[0] + 100*d[1] + 10*d[2] + d[3]
                + 1000*d[4] + 100*d[5] + 10*d[6] + d[1]
                == 10000*d[4] + 1000*d[5] + 100*d[1] + 10*d[7] + d[2]);
        for (int n = 0; n <= 9; n++) {
            if (!used[n]) {
                used[n] = true;
                d[i] = n;
                if (solve(i + 1)) return true;
                used[n] = false;
            }
        }
        return false;
    }
    public static void main(String[] args) {
        if (solve(0)) {
            String letters = "SENDMORY";
            for (int i = 0; i < 8; i++)
                System.out.println(letters.charAt(i) + " = " + d[i]);
            System.out.println("9567 + 1085 = 10652");
        } else {
            System.out.println("No solution.");
        }
    }
```

```
}
```

4. Minimax: Tic-Tac-Toe

```
import java.util.*;
public class Assignment4 {
    static char[] b = new char[9];
    static int[][] lines = {
        {0,1,2},{3,4,5},{6,7,8},{0,3,6},
        {1,4,7},{2,5,8},{0,4,8},{2,4,6}
    };
    static boolean win(char p) {
        for (int[] l : lines)
            if (b[l[0]] == p && b[l[1]] == p && b[l[2]] == p) return true;
        return false;
    }
    static int minimax(boolean oTurn) {
        if (win('O')) return 1;
        if (win('X')) return -1;
        boolean empty = false;
        for (char c : b) if (c == '-') empty = true;
        if (!empty) return 0;
        int best = oTurn ? -2 : 2;
        for (int i = 0; i < 9; i++) {
            if (b[i] == '-') {
                b[i] = oTurn ? 'O' : 'X';
                int score = minimax(!oTurn);
                b[i] = '-';
                best = oTurn ? Math.max(best, score) : Math.min(best, score);
            }
        }
        return best;
    }
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter 9 cells (X, O, -): ");
        b = sc.next().toUpperCase().toCharArray();
        int move = -1, best = -2;
        for (int i = 0; i < 9; i++) {
            if (b[i] == '-') {
                b[i] = 'O';
                int score = minimax(false);
                b[i] = '-';
                if (score > best) { best = score; move = i; }
            }
        }
        if (move == -1) {
            System.out.println("No empty cell.");
        } else {
            b[move] = 'O';
            System.out.println("Board after O's move:");
            for (int i = 0; i < 9; i++)
                System.out.print(b[i] + (i % 3 == 2 ? "\n" : " "));
        }
    }
}
```